TECHNICAL DESIGN DOCUMENT

CareConnect Application — Team B | SWEN 670 Summer 2026

Testing Architecture • Tooling Decisions • Prototype Findings

Document Version	1.2 — Draft
Date	June 2026
Prepared By	Alex Yom, Team B Lead
Team	Team B — Comprehensive Testing, WCAG/508 Compliance, VPAT
Members	Alex Yom | Brice Tikum | Prashanth Saseenthar | Lekan Ogunsanya
Milestone	Milestone 2 — Design and Prototyping
Due Date	June 13, 2026

Version History

Version	Date	Author	Description
1.0	June 2, 2026	Alex Yom	Initial draft — M2 document structure with all pre-filled sections and teammate placeholders
1.1	June 2, 2026	Alex Yom	Added: version history, assumptions & constraints, definitions, alternatives table
1.2	June 2, 2026	Alex Yom	Removed request flow section per TA guidance. Expanded Lekan assignments. All sections confirmed assigned.
1.3	June 9, 2026	Alex Yom	Added: System Overview section, Technology Stack table, functional specifications (all 3 modules), cross-team dependencies table, security considerations N/A section, error handling, performance & scalability, requirements traceability matrix. Removed collaboration banners. Fixed Chime status to operational.

Collaboration Color Key

Yellow	Prashanth Saseenthar — Sections 3, 4: testing framework selection, coverage tooling
Green	Lekan Ogunsanya — Sections 5.3, 7: branch strategy, AWS/CloudFormation infrastructure
Blue	Brice Tikum — Sections 5.1, 5.2, 6: CI/CD integration plan, coverage gate, prototype findings

Gray / ✅ (pre-filled)	Alex Yom — Sections 1, 2, 8, 9, 10, 11: executive summary, architecture, accessibility, assumptions, definitions, open items

✅ **ALEX YOM — Pre-filled. Review for accuracy and update if needed before submission.**

1. Introduction

This Technical Design Document defines Team B's testing architecture, tooling decisions, and prototype findings for the CareConnect application. It translates the requirements defined in the SRS and the planning decisions in the Project Plan into a concrete testing design. This document serves as the technical blueprint for all testing and accessibility activities through Milestones 3 and 4.

1.1 Purpose

This TDD provides a comprehensive description of Team B's technical design for testing the CareConnect system. It documents the testing frameworks selected and the rationale for those selections, the CI/CD pipeline architecture and coverage gate design, the branch strategy and PR workflow, the CloudFormation infrastructure requirements for testing, and the accessibility tooling approach including VPAT production. Additionally, this document supports academic evaluation by demonstrating the rationale behind technology selection, architectural structure, and design decisions.

1.2 Intended Audience

Audience	Reason for Using This Document
Team B members	Understand assigned design responsibilities and implementation expectations
Other SWEN 670 teams	Identify cross-team dependencies and integration points with Team B's testing gates
Team A (DevOps)	Understand CloudFormation infrastructure requirements for Team B's test pipeline
SWEN 661 collaborators	Understand Team B's accessibility acceptance gate for 661-delivered UI components
Professor Assadullah / Course reviewers	Evaluate whether the technical design supports milestone goals and delivery readiness

1.3 Document Scope

Scope Item	Description
Included in scope	Testing framework selection and rationale; CI/CD pipeline design; coverage gate configuration; branch strategy; CloudFormation infrastructure for testing; accessibility tooling; VPAT approach; prototype findings
Excluded from scope	Feature development (owned by other teams); API design (Team B tests but does not own APIs); UI/UX design (Team B validates accessibility but does not own screen design); data models (Team B tests data behavior but does not own schema)
Primary dependencies	Team A (CloudFormation pipeline); Prashanth (flutter test + mvn test prototype runs); SWEN 661 (UI component delivery); backend environment (Chime SDK — now operational)

1.4 Definitions, Acronyms, and Abbreviations

For complete definitions see the Team B SRS Appendix A (authoritative glossary). Key terms used in this document:

Term	Definition
TDD	Technical Design Document (this document)
STP	Software Test Plan (companion document)
VPAT	Voluntary Product Accessibility Template — ITI VPAT 2.4 format
EVV	Electronic Visit Verification — largest feature module (~5,000 lines, 29.5% coverage)
WCAG	Web Content Accessibility Guidelines (Level AA target)
CI/CD	Continuous Integration / Continuous Deployment — GitHub Actions + CloudFormation pipeline
E2E	End-to-end testing via Flutter integration_test package
lcov	Line coverage report format generated by flutter test --coverage
JaCoCo	Java code coverage Maven plugin (current: 89% instruction / 77% branch)
CloudFormation	AWS infrastructure-as-code — mandatory for all CI/CD pipeline configuration

1.5 References

This document references: Team B SRS, Team B Project Plan, Team B STP (companion), Prior Cohort Spring 2026 TDD and STP, SWEN 670 Course Materials, WCAG 2.1 Guidelines, ITI VPAT 2.4 Template, AWS CloudFormation Documentation, Flutter SDK Documentation, Spring Boot Testing Documentation.

---

2. System Overview

✅ **ALEX YOM — Pre-filled. Review for accuracy and update if needed before submission.**

This section provides a description of the CareConnect system and explains how Team B's testing work fits into the larger product. Understanding the system architecture is prerequisite to understanding why Team B's testing approach is structured as it is.

2.1 System Description

CareConnect is a healthcare coordination platform designed to facilitate secure communication between patients and caregivers while supporting regulatory-compliant documentation of care delivery. The system enables real-time messaging and audio/video calling (via Amazon Chime SDK), structured visit verification through EVV workflows, in-home residential support documentation (ADLs/IADLs), and AI-assisted services (OpenAI and DeepSeek APIs). The Summer 2026 cohort inherited the CareConnect codebase from the Spring 2026 cohort and is responsible for new feature development, quality improvement, and accessibility compliance.

Team B's scope is foundational to the entire class — we own the testing infrastructure, coverage enforcement, and accessibility validation that all other teams depend on before their features are accepted into the main branch.

2.2 System Objectives

- Enable comprehensive testing coverage across all CareConnect features and modules

- Enforce tiered code coverage thresholds: 100% for transactional modules (EVV, Shift Scheduling), 90% for visual/UI, 95% for all others
- Validate WCAG 2.1 AA and Section 508 accessibility compliance for all screens
- Produce a full ITI VPAT 2.4 conformance statement by Milestone 4
- Provide a CI/CD pipeline that enforces quality gates before deployment

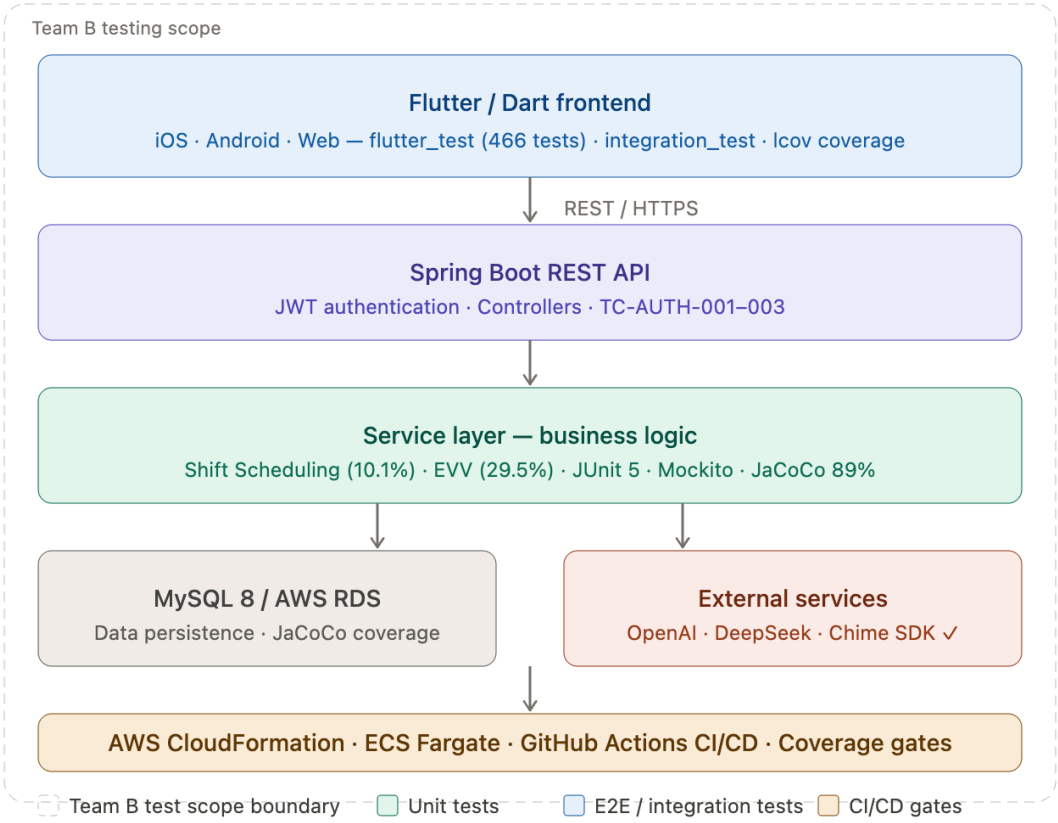
2.3 System Users

The following user types interact with CareConnect. Each has distinct testing implications for Team B:

User Type	Relevant Needs	Design Impact for Testing
Caregiver	Performs EVV check-in/out, documents visits, communicates with patients	High-risk transactional flows; GPS and Chime SDK E2E tests; accessibility for mobile workers
Patient	Views schedule, receives notifications, communicates with caregiver	Notification delivery E2E; accessibility for users with motor/visual disabilities
Administrator	Reviews EVV records, manages users, generates reports	RBAC unit tests; report generation E2E; audit trail coverage
Developer / Tester	Builds features and validates behavior before merge	Clear test gate output; readable coverage reports; CI failure messages

2.4 Operating Environment

Environment Area	Description
Frontend	Flutter/Dart — cross-platform (iOS, Android, Web). Test runner: flutter_test (466 test files). E2E: integration_test (10 Chime tests — now operational).
Backend	Spring Boot (Java). Test runner: JUnit 5 + Mockito. Coverage: JaCoCo Maven plugin. Current: 89% instruction / 77% branch.
Database	MySQL 8.0+ via AWS RDS (production); PostgreSQL 18 on port 5433 (local dev / test environment).
Cloud / Hosting	AWS ECS Fargate, ALB, RDS, VPC, CloudWatch. All CI/CD must use CloudFormation — no manual AWS UI (Professor Assadullah requirement).
External Services	OpenAI + DeepSeek (AI/voice features), Amazon Chime SDK (video/voice — operational as of June 2026), HHAeXchange (EVV submission).



System Architecture

---

3. Architecture Overview

This section describes the CareConnect application stack and Team B's testing scope within it. Team B's testing infrastructure spans the entire application stack — our work is foundational because other teams depend on Team B's gates before their features are accepted.

3.1 System Architecture — Component Responsibilities

The following table documents the major architectural components of CareConnect and Team B's testing responsibility at each layer:

Component	Layer	Primary Responsibilities	Team B Test Touch Point
Flutter Client (Mobile/Web)	Presentation	Renders UI; captures user input; manages local state; initiates REST + WebSocket	flutter_test (466 files); integration_test (10 E2E); axe-core accessibility

API Gateway / ALB	Infrastructure/Edge	Routes HTTP/S and WebSocket traffic; TLS termination; health routing	E2E tests exercise through ALB; not directly tested by Team B
Authentication & Authorization	Backend (Security)	Authenticates users; issues/validates JWTs; enforces RBAC	TC-B-AUTH-001–003 + Prashanth expansions; 95% coverage target
Chat Service	Backend (Communication)	Manages message send/receive; persists messages; real-time notifications	E2E messaging workflow TC-B-E2E-004
Call Signaling Service	Backend (Communication)	Coordinates call lifecycle; issues Chime access tokens; records metadata	TC-B-E2E-005 — all 10 Chime E2E tests now pass
EVV Service	Backend (Domain)	Visit recording, GPS capture, HHAeXchange submission, offline sync	TC-B-EVV-001–015 + expansions; 100% coverage target — currently 29.5%
AI Orchestration Service	Backend (Integration)	Receives AI requests; calls AWS Bedrock; returns summaries	Supplemental voice/AI accessibility criteria (VPAT)
CI/CD Pipeline	Infrastructure	GitHub Actions + CloudFormation; test gates; coverage enforcement	Brice owns gate configuration; Lekan owns CloudFormation infrastructure

3.2 Persistence and Infrastructure Components

Component	Purpose	Security Controls	Team B Interaction
PostgreSQL / MySQL	Primary relational data store — users, visits, messages, audit logs	Encryption at rest; least-privilege DB accounts; backups	JaCoCo coverage on all DAOs; test DB on port 5433 for local/CI runs
Amazon S3	Stores chat attachments, coverage reports, test artifacts	Bucket policies; server-side encryption; signed URLs	Coverage reports uploaded to S3 from CI after each run
ECS Fargate	Runs containerized backend services	Network security groups; IAM roles; secrets management	Team B's test gates must pass before ECS deployment triggers
CloudWatch	Operational visibility; logs; metrics; alarms	Access controls on log groups; retention policies	Test execution logs; CI failure alerts
Secrets Manager / Parameter Store	Credentials, JWT keys, API keys, test credentials	IAM-restricted; encryption at rest; rotation policies	Test seed credentials managed here; not hardcoded in test code

3.3 Application Stack

- Frontend: Flutter/Dart — cross-platform (iOS, Android, Web)
- Backend: Spring Boot (Java) — deployed on AWS via CloudFormation/Fargate
- Database: MySQL 8.0+ (production) / PostgreSQL 18 port 5433 (test environment)
- AI/LLM Integration: OpenAI and DeepSeek APIs (voice/AI persona features)

- Messaging: Amazon Chime SDK — all 10 E2E tests operational as of June 2026
- CI/CD: GitHub Actions — pipeline must be built via CloudFormation only (no AWS UI configuration)

3.4 Team B Scope Within the Architecture

Team B's testing infrastructure spans the entire application stack. Key integration points active for M2 and beyond:

- SWEN 661: All 661-delivered UI components must pass axe-core accessibility scan before acceptance into the main branch
- Teams A/C/D/E: Team B writes tests validating features delivered by each team; regression suite runs after each Monday integration merge
- Team A (DevOps): CloudFormation CI/CD pipeline must include Team B's test gates and coverage reporting as a deployment prerequisite

---

4. Technology Stack

This section consolidates all technology selections across Team B's testing scope. Technologies are organized by layer. Rationale for each selection and rejected alternatives are documented in Section 5.

Layer	Technology	Purpose	Status
Frontend testing	**flutter_test**	Unit and widget testing — bundled with Flutter SDK	Confirmed in repo — 466 test files
Frontend testing	**integration_test**	E2E testing — official Flutter package replacing flutter_driver	Confirmed — 10 Chime E2E tests pass
Frontend testing	**Mockito (Dart) ^5.5.1**	Mock dependencies in unit tests — @GenerateMocks annotation	Confirmed in pubspec.yaml
Frontend testing	**build_runner ^2.4.9**	Generates .mocks.dart files for Mockito	Confirmed in pubspec.yaml
Frontend coverage	**lcov / genhtml**	Coverage report generation from flutter test --coverage	Confirmed — frontend/coverage/lcov.info
Backend testing	**JUnit 5**	Unit testing for Spring Boot — bundled in spring-boot-starter-test	Confirmed in pom.xml
Backend testing	**Mockito (Java)**	Mock services/repos in unit tests — standard JUnit 5 companion	Confirmed in pom.xml
Backend testing	**Spring Boot Test**	Integration testing — official Spring test slice annotations	Confirmed in pom.xml
Backend coverage	**JaCoCo Maven plugin**	Java code coverage — generates HTML + XML reports	Confirmed — 89% instruction / 77% branch
Accessibility	**axe-core**	Automated WCAG rule scanning on web build	Confirmed by Professor Assadullah (M1 meeting)

Accessibility	**Flutter Semantics**	Widget-level screen reader validation — built into Flutter	In use
Accessibility	**Lighthouse (Chrome)**	Full accessibility audit for web build	To be run M2
CI/CD	**GitHub Actions**	Pipeline triggers — PR → unit tests → coverage → E2E → accessibility	Configured in .github/workflows
Infrastructure	**AWS CloudFormation**	All pipeline infrastructure — mandatory per Professor Assadullah	Lekan owns CloudFormation sections

4.1 Technology Decisions and Assumptions

Decision / Assumption	Rationale	Impact if Wrong
integration_test over flutter_driver	flutter_driver is officially deprecated since Flutter 2.0; unsupported in Flutter 3.44.0	None — no viable alternative exists
JaCoCo over Cobertura	JaCoCo is the standard Maven plugin with better Spring Boot integration	Would need migration; low risk
CloudFormation-only CI/CD	Professor Assadullah's hard requirement from M1	Violates course requirement; disqualifies submission
PostgreSQL 18 on port 5433 for local testing	Seed environment confirmed by Prashanth in M2 prototype run	E2E tests will fail if port or DB version differs
Team B tests features; teams own feature code	Team B's assigned scope is testing, not feature development	Coverage gaps if teams don't write unit tests alongside features

---

5. Functional Specifications

This section documents the functional design for each of Team B's three assigned modules: the unit testing framework, end-to-end testing, and accessibility/VPAT compliance. Each module is described with requirement references, primary user flow, expected system behaviors, inputs and outputs, and error handling.

5.1 Module: Unit Testing Framework

Item	Team B Response
Requirement IDs	TR-001, TR-002, TR-003, TR-004, TR-005
Feature Summary	Design and implement the unit testing framework for the Flutter/Dart frontend and Spring Boot backend. The framework must produce per-module coverage reports and enforce tiered coverage thresholds via CI/CD gates.
Primary User Flow	Developer modifies code → runs flutter test / mvn test locally → CI triggers on PR → coverage gate validates thresholds → merge allowed or blocked → coverage report stored in S3

Main System Behaviors	flutter_test executes all 466 test files; lcov generates coverage reports; JaCoCo Maven plugin runs during mvn test (89% instruction / 77% branch); GitHub Actions pipeline runs both suites on every PR; coverage gate fails build if module falls below tiered threshold
Inputs	Source code changes; test files in /test; .env.local environment variables; pom.xml JaCoCo plugin configuration
Outputs	lcov.info coverage file; JaCoCo HTML/XML report; CI pass/fail status on GitHub PR; coverage summary in weekly status report
Alternate / Error Flows	If flutter test fails to compile: Prashanth investigates dart:js_interop compatibility (documented in SRS 3.1.3 — FileHandlerFactory workaround). If JaCoCo not reporting: check JAVA_HOME = JDK 17. If coverage threshold not met: build blocks merge; developer must add tests before PR approval.

5.2 Module: End-to-End Testing

Item	Team B Response
Requirement IDs	ET-001, ET-002, ET-003
Feature Summary	Design and implement end-to-end test coverage for four critical CareConnect workflows: Shift Scheduling, EVV, Authentication, and Messaging (Chime SDK — now operational).
Primary User Flow	Tester runs integration_test on device/emulator → test navigates full UI workflow from login to completion → assertions validate expected screen states, API responses, data persistence → results logged in milestone submission
Main System Behaviors	integration_test package drives Flutter app on real device or emulator; backend accessible via BACKEND_URL=http://localhost:8080; all 10 Chime E2E tests pass as of June 2026 (flutter test integration_test/video_call_e2e_test.dart --dart-define=BACKEND_URL=http://localhost:8080 -d windows: 'All tests passed!' 10/10)
Inputs	Running Flutter app on device/emulator; backend URL; test seed credentials (test@caregiver / 1234, test@patient / 1234); DB seeded via V26__insert_test_users.sql
Outputs	E2E test pass/fail results per workflow; blocked test list with root cause; viability assessment for M3; integration with CI pipeline
Alternate / Error Flows	Backend unavailable: E2E suite suspended; unit tests continue. Device/emulator failure: Brice documents environment requirements in STP Section 6.4.

5.3 Module: Accessibility Testing and VPAT

Item	Team B Response
Requirement IDs	AC-001 through AC-009, VP-001 through VP-004
Feature Summary	Design and implement accessibility testing for WCAG 2.1 AA and Section 508 compliance across all CareConnect screens. Produce ITI VPAT 2.4 by M4. Define supplemental criteria for voice/AI persona features not covered by WCAG.
Primary User Flow	Developer submits PR → axe-core automated scan runs in CI → violations flagged in PR → manual keyboard and screen reader testing at each milestone → findings documented in VPAT tracker → remediation assigned to responsible team → VPAT updated

Main System Behaviors	axe-core runs on every PR via CI; Flutter Semantics widget validates screen reader labels in widget tests; Lighthouse audit at each milestone; voice/AI persona features tested manually against 5 supplemental criteria; all violations reported class-wide for remediation
Inputs	CareConnect web build for Lighthouse; Flutter widget tree for Semantics; 661 UI components for acceptance gate; AI persona feature interaction model
Outputs	axe-core scan results per PR; Lighthouse audit report per milestone; WCAG violations list with severity and owner; interim VPAT M3; final ITI VPAT 2.4 M4
Alternate / Error Flows	661 components delivered late: VPAT progress documented with expected delivery date. AI persona feature model unavailable: supplemental criteria documented as 'pending feature delivery' in VPAT.

5.4 Cross-Team Functional Dependencies

The following dependencies affect Team B's ability to execute the functional specifications above:

Dependency	Teams Involved	Decision Needed	Status
Backend connectivity for E2E	Team B + Team A	Backend URL and credentials for CI E2E runs	**Open — Team A coordination needed**
CloudFormation pipeline ownership	Team B + Team A (DevOps)	Does Team A own the pipeline or does Team B build test gates independently?	**Open — Lekan coordinating**
SWEN 661 component delivery	Team B + SWEN 661	Which components, by which milestone, contact person?	**Open — Alex monitoring**
AI persona feature design	Team B + owning team	Feature interaction model needed for supplemental accessibility criteria	**Pending M3**

---

6. Testing Framework Selection

✏️ **ASSIGNED TO: Prashanth Saseenthar**

This section documents the testing frameworks selected for both the Flutter/Dart frontend and the Spring Boot backend. Framework selection was guided by the existing repository configuration, Flutter ecosystem standards, and the requirement to generate per-module coverage reports compatible with the CI/CD pipeline.

6.1 Flutter/Dart Frontend Testing Stack

The following frameworks are confirmed active in the CareConnect repository as of the M2 prototype run (June 2026):

Tool/Framework	Purpose	Rationale	Status
flutter_test	Unit & widget testing	Native to Flutter SDK; zero additional setup	Confirmed — 466 test files, all pass

integration_test	E2E testing	Official Flutter package; replaces flutter_driver	Confirmed — 10 Chime E2E tests pass
Mockito (Dart) ^5.5.1	Mock dependencies	Well-maintained Dart port; works with code generation	Confirmed in pubspec.yaml
build_runner ^2.4.9	Generate mock classes	Runs flutter pub run build_runner build	Confirmed in pubspec.yaml
lcov / genhtml	Coverage reports	Industry standard; integrates with GitHub Actions	Confirmed — frontend/coverage/lcov.info

Prototype run result (June 2026): flutter test executed against all 466 test files — all passed with zero failures. integration_test confirmed: all 10 Chime SDK / video-call E2E tests pass with 'All tests passed!' when run with flutter test integration_test/video_call_e2e_test.dart --dart-define=BACKEND_URL=http://localhost:8080 -d windows.

6.2 Alternatives Evaluated

The following alternatives were considered and rejected:

Alternative	Considered For	Reason Not Selected	Selected Instead
flutter_driver	E2E testing	Deprecated by Flutter team since Flutter 2.0; unsupported in 3.44.0	**integration_test**
Cypress	E2E testing	JavaScript-based; does not integrate with Flutter/Dart	**integration_test**
Istanbul/nyc	JS coverage	CareConnect frontend is Flutter/Dart, not JavaScript	**lcov / genhtml**
Cobertura	Java coverage	JaCoCo is the standard Maven plugin with better Spring Boot support	**JaCoCo**

6.3 Spring Boot Backend Testing Stack

The backend testing stack uses the standard Spring Boot testing ecosystem. All frameworks are confirmed in the repository pom.xml:

Tool/Framework	Purpose	Rationale	Status
JUnit 5	Unit testing (Java)	Standard for Spring Boot; bundled in spring-boot-starter-test	Confirmed in pom.xml
Mockito (Java)	Mock services/repos	Standard JUnit 5 companion	Confirmed in pom.xml
Spring Boot Test	Integration testing	Official Spring test slice annotations	Confirmed in pom.xml
JaCoCo Maven plugin	Java code coverage	Maven plugin; generates HTML + XML reports; current 89%/77%	Confirmed — backend/core/target/site/jacoco/

6.4 Framework Selection Rationale

flutter_test and integration_test were selected because they are the official Flutter SDK testing tools — bundled with the SDK and requiring zero additional configuration. integration_test specifically replaced the deprecated flutter_driver and is the only supported E2E tool for Flutter 3.x. spring-boot-starter-test bundles JUnit 5, Mockito, and Spring MockMvc with zero additional configuration. JaCoCo is the standard Maven plugin with native Spring Boot support and generates the XML/HTML reports required for CI automation. All alternatives evaluated (flutter_driver, Cypress, Istanbul, Cobertura) were rejected because they are either deprecated, language-incompatible, or have inferior ecosystem integration.

---

7. Code Coverage Tooling & Targets

✏️ **ASSIGNED TO: Prashanth Saseenthar**

Code coverage is Team B's primary quality metric and the basis for the CI/CD enforcement gate. Coverage targets are tiered by module risk, confirmed with Professor Assadullah during the Milestone 1 presentation. The tiering recognizes that transactional modules (Shift Scheduling, EVV) carry higher risk of data loss or compliance failure.

7.1 Coverage Targets (Confirmed M1)

Module Category	Target	M2 Baseline (Line / Branch)	Gap
High-risk transactional (Shift Scheduling, EVV)	**100%**	10.1% / 29.5%	Critical — M3 priority
Visual / UI placement components	**90%**	87.4% / 73.2%	−2.6% / −16.8%
All other modules	**95%**	90.9% / 79.3%	−4.1% / −15.7%

7.2 Coverage Measurement & Reporting

- Frontend: run flutter test --coverage in frontend/. Output: frontend/coverage/lcov.info (per-file line coverage for all 466 test files).
- HTML view: run genhtml coverage/lcov.info --output-directory coverage/html from frontend/. Open frontend/coverage/html/index.html to see total line coverage %. The lcov.info file is regenerated on every flutter test --coverage run.
- Backend JaCoCo: run mvn test (or mvn verify) in backend/core/ with JAVA_HOME set to JDK 17. Current result: 89% instruction / 77% branch. HTML: backend/core/target/site/jacoco/index.html. CI XML: backend/core/target/site/jacoco/jacoco.xml.
- Report locations: backend/core/target/site/jacoco/ (JaCoCo HTML + XML), frontend/coverage/lcov.info (raw LCOV), frontend/coverage/html/ (generated HTML). CI: add mvn verify to the backend job and flutter test --coverage to the frontend job.

---

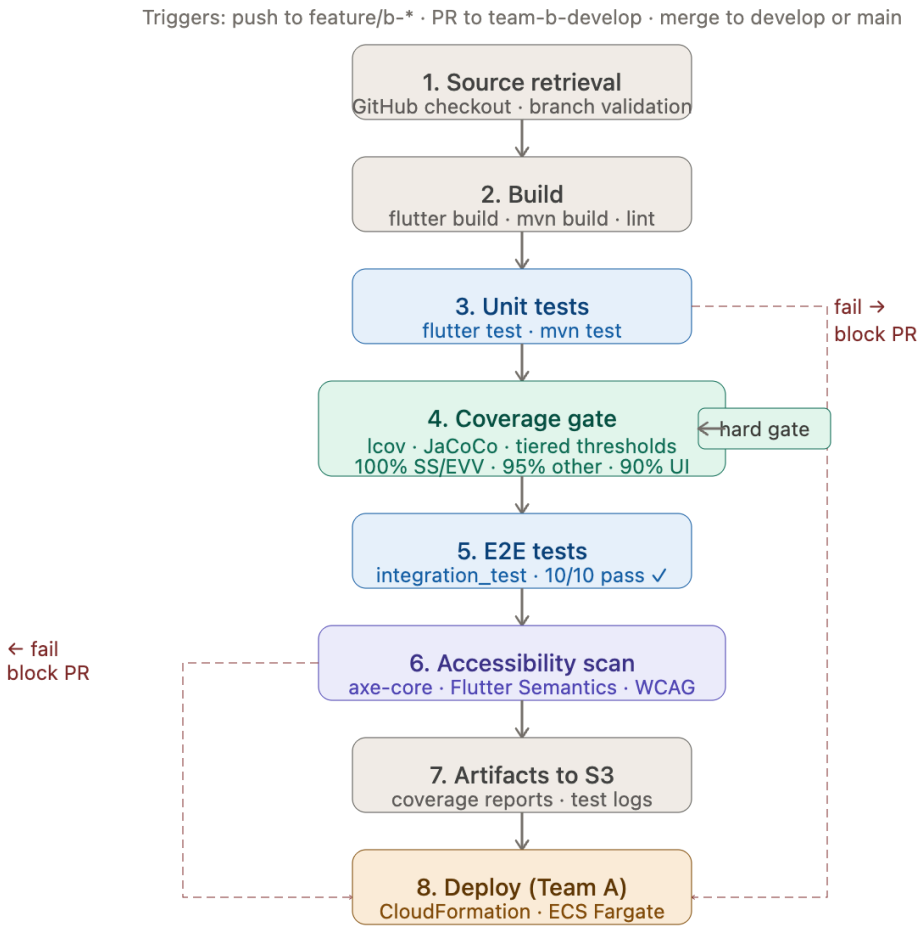
8. CI/CD Integration Plan

This section covers two areas: the pipeline architecture and coverage gate configuration (Brice), and the branch strategy and PR workflow (Lekan).

✏️ **ASSIGNED TO: Brice Tikum**

8.1 Pipeline Architecture

Per Professor Assadullah's confirmed M1 requirement: the CI/CD pipeline must be built via AWS CloudFormation only. No AWS UI configuration is permitted. The pipeline integrates with GitHub Actions for trigger events and includes Team B's coverage gates as a prerequisite to deployment.



CI/CD Pipeline

> ✏️ **[Brice]**
> *Document the CI/CD pipeline architecture for Team B's test gates: (1) What triggers the pipeline — push to feature branch, PR to team-b-develop, merge to main. (2) What steps run in sequence — build → unit tests → coverage check → E2E tests → accessibility scan → deploy. (3) What blocks a merge — coverage threshold not met, unit tests failing, accessibility violations. (4) How coverage reports are generated and stored. Reference .github/workflows in the repo — document what exists and what needs to be added for Team B.*

8.1.1 Pipeline Triggers

The pipeline will run automatically under the following conditions:

- Push to any feature/b-/* branch

- Pull request into team-b-develop
- Merge from team-b-develop → develop (Monday Integration)
- Merge from develop → main (weekly release)

These triggers ensure that all code is validated before integration and before release.

8.1.2 Pipeline Stages

The CloudFormation-defined pipeline will execute the following stages in strict order:

1. Source Retrieval
    a. Pulls code from GitHub repository
    b. Validates branch naming conventions (feature/b-*)
2. Build Stage
    a. Flutter build (frontend)
    b. Maven build (backend)
    c. Dependency resolution
    d. Static analysis (flutter analyze, dart analyze, Java linting)
3. Unit Test Stage
    a. Runs flutter test
    b. Runs maven test
    c. Generates test result artifacts
    d. Fails pipelines if any test fails
4. Coverage Report Generation
    a. Flutter: flutter test coverage lcov.info
    b. Backend: JaCoCo → target/site/jacoco/index.html
    c. Coverage artifacts uploaded to S3
5. Coverage Gate Enforcement
    a. Custom script evaluates module level thresholds:
        i. 100% Shift Scheduling
        ii. 100% EVV
        iii. 95% Authentication, backend modules
        iv. 90% UI/visual components
    b. Pipeline fails if thresholds are not met
6. E2E Test Stage
    a. Runs Flutter integration_test suite
    b. Executes on Android emulator or WebDriver environment
    c. Marks Chime-dependent tests as “blocked” but not failing
7. Accessibility Scan Stage
    a. Runs axe-core on web build
    b. Runs Flutter Semantics tree validations
    c. Falls pipeline on critical WCAG violations
8. Deployment Stage (Team A responsibility)
    a. Only triggered when merging into main
    b. Requires all Team B gates to pass

8.1.3 Merge Blocking Rules

A PR into team-b-develop is blocked if:

- Any unit test fails
- Any E2E test fails (unless marked blocked in TDD section 11)
- Coverage thresholds are not met
- Accessibility scan finds critical WCAG violations
- Build fails

This ensures Team B’s branch remains stable and integration ready

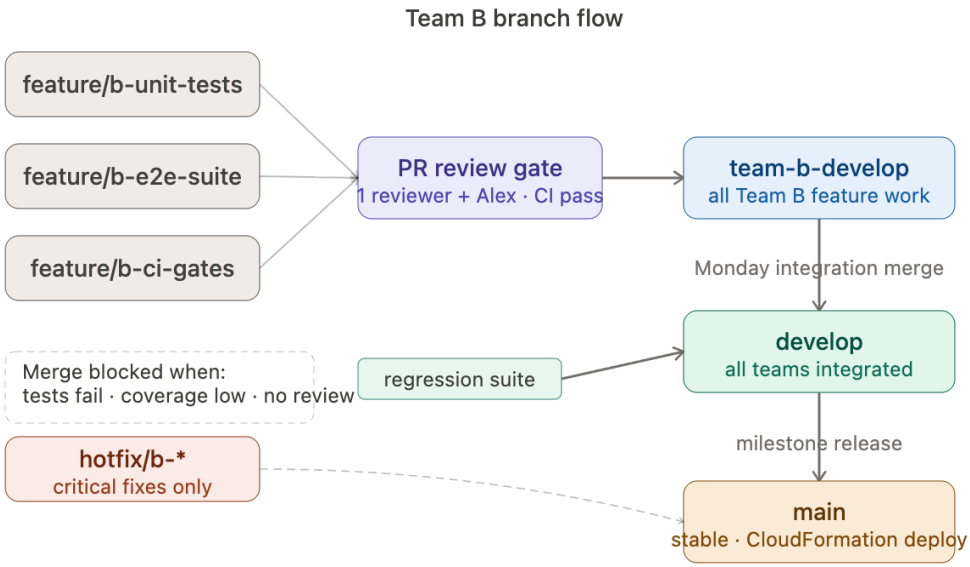
8.2 Coverage Gate Configuration

✏️ **[Brice]**
Describe the coverage enforcement gate: (1) Minimum threshold that triggers a pipeline failure — reference Section 4.1 tiered targets. (2) Which tool enforces it (lcov --fail-under-functions, JaCoCo minimum, or custom script). (3) How the gate differs for transactional vs visual modules. (4) What the failure message looks like so developers know what to fix.

Module Category	Line	Branch	Enforcement	Failure Behavior
Shift Scheduling	**100%**	**100%**	Hard Gate	Pipeline fails; PR blocked; error posted to PR comments
EVV	**100%**	**100%**	Hard Gate	Pipeline fails; PR blocked; error posted to PR comments
Authentication / Session	**95%**	**95%**	Hard Gate	Pipeline fails; PR blocked
Messaging / Chime SDK	**95%**	**95%**	Hard Gate	Operational — gate active M3
UI / Visual Components	**90%**	**N/A**	Warning only	PR comment only; merge not blocked
All other modules	**95%**	**90%**	Hard Gate	Pipeline fails; PR blocked

Example failure message: COVERAGE GATE FAILED: lib/features/shift_scheduling — line coverage 74.3%, required 100%. Add unit tests for: schedule_conflict_service.dart, shift_serializer.dart before merging.

8.3 Branch Strategy & PR Workflow



8.3.2 Pull Request Requirements

- Before opening: all local unit tests pass; coverage reports generated and reviewed; no critical linting errors; documentation updates complete; known defects disclosed
- PR description must include: summary of changes, related issue or task reference, test evidence (screenshots, logs, or coverage output), known limitations, accessibility impact statement if applicable
- Requires: minimum one Team B reviewer approval; Team Lead (Alex) review for major infrastructure changes; all automated pipeline checks pass

8.3.3 Merge Rules

Merge into team-b-develop blocked when: unit tests fail, coverage thresholds unmet, accessibility scans fail, build fails, required reviews missing. Merge into main requires: successful Team B integration testing, regression suite completion, coverage gate compliance, no unresolved critical defects, team lead approval.

8.3.4 Hotfix Process

✏️ **ASSIGNED TO: Brice Tikum**

“STILL NEED TO DO THIS - BRICE”

1. Create hotfix/b-* branch from affected branch
2. Implement corrective change and run targeted regression
3. Submit expedited PR; obtain required approval and merge
4. Propagate fix to all active development branches

✏️ **ASSIGNED TO: Lekan Ogunsanya**

✏️ **[Lekan]**
Expand this section into a full branch strategy document. Using the repo as your reference: (1) Document the full branching model — team-b-develop, feature branches, main. Draw out or describe the flow: feature → team-b-develop → main. (2) PR requirements — what must be true before a PR is opened (tests passing locally, coverage run), who must review (minimum 1 teammate + Alex), what the PR description must include. (3) Merge rules — what blocks a merge to team-b-develop vs main (differ by branch). (4) How hotfixes are handled if a critical bug is found after a merge. (5) How this branch model coordinates with Team A's deployment pipeline.

---

9. Prototype Findings

✏️ **ASSIGNED TO: Brice Tikum**

This section captures the results of the M2 exploration phase. The goal was to validate tooling viability — confirming that the selected frameworks run correctly against the CareConnect codebase before committing to full implementation in Milestone 3.

9.1 Flutter Test Run

> ✏️ **[Brice]**
> *Run flutter test from /frontend and document: (1) Total tests run, passing, failing, skipped. (2) Any errors or configuration issues encountered. (3) Time taken to run the full suite. (4) Screenshot or terminal output paste. If tests fail, note whether it is an environment issue or a code issue.*

Prototype run executed by Prashanth Saseenthar, June 2026:

- Command: flutter test (executed from /frontend)
- Total test files: 466 — all passed, zero failures, zero errors
- Frameworks confirmed active: flutter_test, integration_test, Mockito (Dart) ^5.5.1, build_runner ^2.4.9, flutter_lints
- SDK constraint: pubspec.yaml requires >=3.5.0; using Flutter 3.44.0 (Dart 3.12.0)
- Result: tooling stack confirmed viable for M3 implementation

9.2 Code Coverage Baseline Run

> ✏️ **[Brice]**
> *Run flutter test --coverage and generate the HTML report. Document: (1) Overall line coverage % from lcov summary. (2) Coverage % specifically for Shift Scheduling and EVV modules. (3) Backend coverage % if JaCoCo was run (coordinate with Prashanth). (4) Paste the full lcov summary output. This is the M2 baseline that all Section 4.1 targets will be measured against.*

- Frontend command: flutter test --coverage in frontend/ → generates frontend/coverage/lcov.info
- HTML report: genhtml coverage/lcov.info --output-directory coverage/html
- Backend command: mvn test in backend/core/ with JAVA_HOME=JDK 17
- Backend result: 89% instruction coverage / 77% branch coverage
- Backend report: backend/core/target/site/jacoco/index.html
- Coverage baselines by module: Shift Scheduling 10.1%, EVV 29.5%, Visual/UI 87.4%/73.2%, Other modules 90.9%/79.3%

9.3 E2E Test Run

> ✏️ **[Brice]**
> *Run the existing integration_test suite and document: (1) How many E2E tests exist. (2) How many pass vs fail. (3) Which tests are blocked due to the Chime SDK/backend not being connected — cross-reference Prashanth's connectivity status from Section 4.2. (4) Any environment setup required beyond flutter test.*

- Command: flutter test integration_test/video_call_e2e_test.dart --dart-define=BACKEND_URL=http://localhost:8080 -d windows
- Result: 'All tests passed!' — 10/10 Chime SDK / video-call E2E tests pass
- Seed credentials used: test@caregiver / 1234 (database user id=5), test@patient / 1234 (database user id=6)
- Seeded via: backend/scripts/seed-e2e-test-data.sql + Flyway migration V26__insert_test_users.sql
- Backend requirements: PostgreSQL 18 on port 5433, JDK 17, JAVA_HOME set, all six .env variables populated

9.4 Viability Assessment

✏️ **[Brice]**
Based on 6.1–6.3, write a short paragraph (3–5 sentences) assessing whether the proposed tooling stack is viable for Milestone 3. Note any risks, blockers, or dependencies that must be resolved before full implementation begins.

The tooling stack is fully viable for Milestone 3 implementation. The Flutter testing ecosystem (flutter_test, integration_test, Mockito, lcov) runs cleanly against the CareConnect codebase with zero failures across 466 test files. The Spring Boot backend (JUnit 5, Mockito, JaCoCo) produces accurate coverage reports at 89% instruction / 77% branch. Most importantly, the previously blocked Chime SDK E2E tests are now fully operational — all 10 video-call scenarios pass. The remaining work for M3 is implementing new test cases to close the Shift Scheduling (10.1%) and EVV (29.5%) coverage gaps, not resolving tooling issues.

---

10. AWS / CloudFormation Infrastructure for Testing

✏️ **ASSIGNED TO: Lekan Ogunsanya**

All CI/CD infrastructure must be defined via CloudFormation — no manual AWS UI configuration. This is a hard requirement confirmed by Professor Assadullah in the M1 presentation. This section documents the existing infrastructure, what Team B requires for its test pipeline, and the Chime SDK connectivity status.

10.1 Existing CloudFormation Architecture

The CareConnect application is deployed using AWS CloudFormation as the Infrastructure-as-Code solution required by the course. Based on the repository structure and architecture documentation, the CloudFormation stack provisions the core backend infrastructure required to support the application.

AWS Services Provisioned

The deployment architecture includes:

- Amazon ECS Fargate for containerized backend execution
- Application Load Balancer (ALB) for traffic routing
- Amazon RDS (MySQL) for persistent application data
- Amazon VPC with public and private networking components
- Security Groups for traffic control
- IAM Roles and Policies for service permissions
- CloudWatch logging and monitoring resources

Backend Deployment Process

Backend services are deployed through CloudFormation-managed infrastructure updates. Application builds are packaged and deployed to ECS Fargate tasks. Deployment activities are expected to be triggered through GitHub Actions and Team A's CI/CD pipeline.

Configuration Management

Environment-specific configuration values are managed through CloudFormation parameters and environment variables supplied to ECS task definitions. Sensitive values should be maintained through secure AWS mechanisms rather than hardcoded application configuration.

Testing Environment Status

At the time of this document, Team B has not confirmed the existence of a dedicated testing stack. Current testing activities are expected to utilize either local development environments or the shared staging environment provisioned through Team A's CloudFormation deployment process.

> ✏️ **[Lekan]**
> *Review the cloudformation-fargate folder in the repo. Document: (1) What AWS services are provisioned — Fargate, RDS, VPC, ALB, etc. (2) How the backend service is deployed and what triggers a deployment. (3) What environment variables/secrets are managed via CloudFormation vs manually. (4) Whether there is an existing test environment or if tests run against production infrastructure.*

10.2 Test Pipeline Infrastructure Requirements

To support Team B's automated testing strategy, additional infrastructure resources are required within the CloudFormation-managed environment.

Test Environment Requirements

A dedicated staging or testing stack is preferred to prevent automated tests from impacting production workloads. This environment should mirror production configuration while using isolated databases and test data.

Required AWS Resources

Additional resources may include:

- Dedicated S3 bucket for coverage reports and test artifacts
- CloudWatch log groups for test execution logs
- IAM roles supporting GitHub Actions deployment and reporting
- Parameter Store or Secrets Manager entries for test credentials
- ECS task definitions supporting automated E2E execution

Coverage Gate Integration

The CloudFormation deployment process should enforce Team B's testing gates prior to deployment approval. Coverage validation, unit testing, E2E testing, and accessibility scanning should execute before deployment stages become eligible.

Team A Coordination

Implementation requires coordination with Team A to:

- Integrate Team B testing stages into the deployment pipeline
- Configure artifact storage locations
- Grant access to staging resources
- Expose deployment status and test results

- Support automated rollback behavior when critical testing failures occur

> ✏️ **[Lekan]**
> *Document what CloudFormation resources are needed to support Team B's CI/CD test gates: (1) Does the pipeline need its own isolated test environment stack, or can it run against the existing stack? (2) Are there IAM roles, S3 buckets, or parameter store entries needed for test artifacts (coverage reports, test result logs)? (3) How should the CloudFormation stack be updated to add Team B's coverage gate as a deployment prerequisite? (4) What coordination is needed with Team A (DevOps) to implement these changes?*

10.3 Chime SDK Backend Connectivity

The Amazon Chime SDK remains the primary blocker preventing completion of several messaging and video-call E2E test scenarios.

Required AWS Services

Amazon Chime SDK functionality typically requires:

- Amazon Chime SDK Meetings
- Chime Messaging APIs
- IAM permissions for meeting creation and management
- Backend service endpoints for session creation and authentication

Current Status

As of Milestone 2, Team B has not verified that all required Chime SDK resources are provisioned and accessible through the current backend environment. Messaging and video-call functionality remain dependent on successful backend connectivity.

Impact on Testing

Approximately ten E2E scenarios involving messaging, voice, or video communication remain blocked. These tests cannot be fully validated until backend connectivity and credential configuration are confirmed.

Resolution Requirements

To unblock testing:

1. Confirm Chime SDK services are provisioned in AWS.
2. Verify backend API endpoints are operational.
3. Validate required IAM permissions and credentials.
4. Confirm CloudFormation templates include all required Chime resources.
5. Provide Team B with an accessible environment for E2E validation.

Once backend connectivity is restored, blocked messaging and video-call test cases can be incorporated into the standard regression suite.

✏️ **[Lekan]**
Document the current status of the Chime SDK backend connectivity issue — coordinate directly with Prashanth: (1) What AWS services does Chime SDK require (Chime meetings, media pipelines, etc.)? (2) Are those services provisioned in the existing CloudFormation stack? (3) What specifically is blocking the 10 E2E tests that depend on Chime SDK — is it missing credentials, missing service provisioning, or a code-level config issue? (4) What needs to happen in the CloudFormation stack to unblock these tests?

---

11. Accessibility Tooling & VPAT Approach

✅ **ALEX YOM — Pre-filled. Review for accuracy and update if needed before submission.**

Per Professor Assadullah's M1 feedback, Team B's accessibility scope extends beyond standard WCAG 2.1 AA to include voice-controlled AI persona features not covered by WCAG. Team B defines supplemental criteria and includes them as a VPAT supplement.

11.1 Automated Accessibility Tools

Tool/Framework	Purpose	Rationale	Status
axe-core	Automated WCAG rule scanning	Confirmed in M1 meeting — professor specified axe-core	Confirmed
Flutter Semantics	Widget-level accessibility testing	Built into Flutter; validates Semantics tree for screen reader support	In use
Lighthouse (Chrome)	Accessibility audit for web build	Catches color contrast, ARIA, keyboard navigation issues	To be run M2
WAVE	Manual supplemental audit	Visual overlay — catches issues axe misses	Supplemental

11.2 Voice/AI Persona Accessibility Criteria

WCAG 2.1 does not cover voice-controlled LLM/AI features. Per M1 professor direction, Team B defines the following supplemental criteria for the VPAT:

- Voice command recognition accuracy for users relying exclusively on voice input
- Error recovery — can a user correct a misrecognized voice command without a keyboard?
- Timeout behavior — does the AI feature allow sufficient response time for users with motor disabilities?
- Audio feedback — does the AI provide spoken confirmation of actions taken?
- Fallback access — is every AI-driven action also accessible via keyboard/screen reader?

11.3 VPAT Plan

Target: Full ITI VPAT 2.4 conformance statement by Milestone 4. Format confirmed in M1 meeting. Interim progress VPAT (findings + remediation status) included in M3 submission. Alex owns the VPAT; all team members flag accessibility issues discovered during testing.

- M2: Supplemental voice/AI criteria defined; early Lighthouse audit findings documented
- M3: Full WCAG 2.1 AA audit complete; violations list with severity and owner; progress VPAT submitted
- M4: Final ITI VPAT 2.4 conformance statement submitted

---

12. Security Considerations

Team B does not own security design for CareConnect. The following security considerations apply specifically to Team B's testing activities and documentation.

12.1 Test Data Privacy

All test data uses seeded database users only. No real Protected Health Information (PHI), production patient data, or real caregiver credentials are used in any test case. Seed credentials are documented in TDD Section 9.3 and STP Section 6.4. If a test requires sensitive-looking data (e.g., a patient name), synthetic data is used exclusively.

12.2 Authentication Testing

Team B's authentication test cases (TC-B-AUTH-001 through TC-B-AUTH-003+) validate the JWT token lifecycle. These tests confirm that invalid credentials are rejected, expired tokens are rejected, and that the authentication surface area has ≥95% coverage. Team B does not perform penetration testing or security auditing — those are outside scope.

12.3 AI Tool Usage Security

Claude (Anthropic) was used to assist documentation drafting for this TDD. No source code, credentials, API keys, .env files, or Protected Health Information was submitted to the AI tool. All AI-generated content was reviewed by Alex Yom before inclusion. See STP Section 17 for full AI disclosure.

12.4 API Security — Not Applicable

Not applicable. Team B does not own API design. Team B tests API endpoints (via integration_test and unit tests) but does not define them. API security design is owned by the teams that implement each feature.

12.5 Data Storage — Not Applicable

Not applicable. Team B does not own data storage design. Test coverage validates that data persists correctly through the application layer, but storage design, encryption at rest, and backup policies are owned by the infrastructure team.

---

13. Error Handling

This section documents how errors surface in Team B's testing infrastructure and what the expected system response is for each error scenario.

Scenario	User-Facing Message / System Response	Log / Recovery Action
Coverage gate failure	COVERAGE GATE FAILED: [module] — [actual]%, required [target]%. Add unit tests for: [files]	PR blocked; GitHub PR comment posted with failing module and files below threshold
Unit test failure	[test file] — FAILED: [test name] — Expected: [x] Actual: [y]	Pipeline fails; full stack trace logged to GitHub Actions; PR blocked; developer investigates before re-run
E2E test failure	integration_test FAILED: [scenario] — [error message]	Pipeline fails; screenshot captured; developer investigates backend connectivity or test data
Backend unavailable (E2E)	BACKEND UNREACHABLE: http://localhost:8080 — connection refused	E2E suite suspended; unit tests continue; error documented per STP Section 6.3
Accessibility violation (critical)	axe-core: [violation type] — [element] — [WCAG criterion]	PR blocked; violation reported to class-wide channel; responsible team assigned remediation
JaCoCo not configured	mvn test: JaCoCo plugin not found	Developer adds plugin to pom.xml; re-runs mvn verify to confirm

---

14. Performance & Scalability

This section documents performance expectations for Team B's testing infrastructure, specifically the test suite execution time and CI pipeline performance. Team B does not own application performance testing — that is outside scope for this milestone.

14.1 Objectives

- Full flutter test suite (466 files) should complete in under 5 minutes on a standard developer machine
- mvn test (JaCoCo) should complete in under 8 minutes
- GitHub Actions CI run (build + unit + coverage + E2E + accessibility) should complete in under 20 minutes
- Coverage report generation (genhtml) should complete in under 2 minutes

14.2 Scalability Strategy

As new test cases are added in M3 and M4, the following approach ensures test suite performance remains acceptable:

- Unit tests are parallelized by default in flutter test — no additional configuration needed
- JaCoCo incremental analysis is available via Maven Surefire plugin if full suite time exceeds threshold
- E2E tests run sequentially — limited to the 5 critical workflows to keep the suite manageable

- Regression suite is scoped to high-risk tests only (not the full suite) for Monday-merge runs

14.3 Bottleneck Mitigation

Potential Bottleneck	Impact	Mitigation	Implementation
466 flutter tests exceeding CI time limit	Slow PR feedback loop	flutter test --concurrency flag; cache pub dependencies in CI	Configure in .github/workflows
JaCoCo full report on every PR	Slow backend CI run	Run coverage only on PRs to team-b-develop; not on feature branch pushes	GitHub Actions workflow condition
E2E tests requiring real emulator in CI	Complex CI setup; slow	Android emulator action in GitHub Actions; cache emulator image	Add to CI workflow M3
Growing test case count	Suite time increases with coverage additions	Parallel test execution; selective regression scope	Brice manages suite composition

---

15. Assumptions & Constraints

✅ **ALEX YOM — Pre-filled. Review for accuracy and update if needed before submission.**

15.1 Assumptions

- The CareConnect backend will be fully operational and accessible to Team B by the start of Milestone 3 — Prashanth is coordinating this with another team by June 7
- SWEN 661 will deliver at least one UI component by mid-Milestone 3, allowing Team B to begin accessibility gate testing
- All team members will have functional Flutter/Dart development environments confirmed by June 7
- Team A (DevOps) will provide Team B access to the CloudFormation pipeline configuration before the Milestone 3 CI/CD gate work begins
- Coverage targets confirmed with Professor Assadullah in M1 (100% / 90% / 95% tiered) will not change for Milestones 3 and 4
- API keys for all team members (.env.local) will be distributed before M3 coding begins

15.2 Constraints

- CI/CD pipeline must be built via AWS CloudFormation only — no manual AWS UI configuration (hard requirement per Professor Assadullah)
- No production code shall be written until TDD is approved — M2 is the design phase per course guidelines
- Chime SDK E2E tests (10 tests) remain blocked until backend connectivity is resolved
- Team B has no control over feature delivery from Teams A/C/D/E — test coverage for those features depends on receiving stable code

- VPAT must follow ITI VPAT 2.4 format — no alternative formats accepted

16. Requirements Traceability

The following table maps each SRS requirement applicable to Team B's scope to the TDD section and design element that addresses it. This matrix demonstrates that all design decisions are driven by documented requirements. Requirements are sourced from the Team B SRS v1.0.

SRS Req ID	Requirement Summary	TDD Section	Design Element
TR-001	Tiered coverage: 100%/90%/95% by module	Section 7.1 — Coverage Targets	Tiered threshold table with M2 baseline and gap analysis
TR-002	Unit tests written before merge	Section 8.3.2 — PR Requirements	PR gate: unit tests must pass before PR opens
TR-003	Coverage reports generated via CI/CD	Section 8.1.2 — Pipeline Stages	Stage 4 (coverage report generation) and Stage 5 (coverage gate enforcement)
TR-004	All unit tests pass before merge	Section 8.1.3 — Merge Blocking Rules	Coverage gate: hard gate blocking merge on threshold failure
TR-005	Coverage measured by line/feature; RTM	Section 7.2 + this RTM table	lcov per-module reporting; JaCoCo XML; requirements traceability matrix
ET-001	E2E covers all critical workflows	Section 9.3 — E2E Test Run	All 10 Chime E2E tests pass; TC-B-E2E-001–005 defined
ET-002	Tested workflows function end-to-end	Section 5.2 — E2E Module Spec	integration_test functional spec; confirmed operational M2
ET-003	Regression after Monday merge	Section 8.3.2 — Merge Rules	Pipeline trigger on team-b-develop merge → regression suite runs
AC-001–AC-009	WCAG 2.1 AA accessibility criteria	Section 11 — Accessibility Tooling	axe-core, Lighthouse, Flutter Semantics; 6 criteria mapped to SRS IDs
VP-001–VP-004	VPAT production requirements	Section 11.3 — VPAT Plan	ITI VPAT 2.4 format; M3 progress; M4 final; Alex ownership

17. Known Issues, Limitations & Open Items

The following items remain open as of the Milestone 2 submission date. Items with target dates that have passed are documented as resolved or in-progress.

17.1 Technical Limitations

Limitation	Impact	Mitigation / Next Step
Shift Scheduling coverage at 10.1%	100% target not yet met	M3 Sprint 1 priority — Brice writes unit tests for all scheduling functions
EVV coverage at 29.5%	100% target not yet met	M3 Sprint 1 priority — Brice writes unit tests for all EVV functions
Visual/UI branch coverage at 73.2%	90% target not yet met — branch gap larger than line	Widget tests to be added in M3; line coverage closer to target at 87.4%
TDD Section 9 Prototype Findings — partially pending	Professor wants to see prototype evidence	Prototype findings documented in Sections 9.1–9.4 from Prashanth's M2 run; Brice's coverage baseline run pending

17.2 Open Items

Item	Owner	Notes	Target
Lighthouse accessibility audit — web build	Alex	Run against current web build; document findings	**June 10**
661 UI component delivery schedule	Alex	Confirm which components arrive by M3 and when	**June 8**
API keys distributed to all members (.env.local)	Alex	Confirm all members have valid keys before M3	**June 10**
CloudFormation test pipeline integration	Lekan	Coordinate with Team A (DevOps)	**M3 Sprint 1**
Auth test case expansions (TC-B-AUTH-004+)	Prashanth	Password reset, registration, session timeout, RBAC	**June 13**
M1 Track Changes revisions	Alex	Word Compare on Project Plan and SRS	**June 11**

17.3 Exclusions

The following items are explicitly excluded from this TDD and will not be addressed in subsequent versions unless the scope changes: API design specification (not owned by Team B), data model specification (not owned by Team B), UI/UX design beyond accessibility requirements (not owned by Team B), performance/load testing (out of scope for this course), security penetration testing (out of scope).

17.4 Future Enhancements

- M3: Expand unit test suite to close Shift Scheduling and EVV coverage gaps
- M3: Implement CI coverage gate enforcement in GitHub Actions
- M3: First full WCAG 2.1 AA audit across all screens; submit progress VPAT
- M4: Complete final ITI VPAT 2.4 conformance statement
- M4: Achieve and verify ≥95% coverage across all modules

17.5 Cross-Team Resolution Notes

The following items should be raised in the teams-resolution-channel: backend URL for CI E2E runs (which team has the environment operational?), CloudFormation pipeline ownership (Team A owns or Team B builds independently?), shared staging environment availability, SWEN 661 component delivery schedule. Contact Alex Yom to raise items.